

Abstraction and Inheritance

- In this topic, we solve issues that we experienced in the previous topics.
- We discuss how **Abstraction** and **Inheritance** can be used to refactor code smells.

Group things that belong together

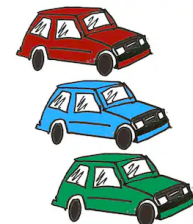
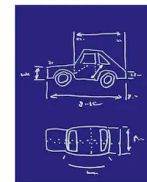
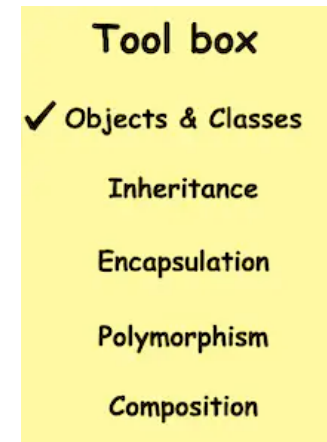
Vera	2000
Chuck	1800
Samantha	1800
Roberto	2100
Dave	2200
Tina	2300
Ringo	1900

Vera	2000
Chuck	1800
Samantha	1800
Roberto	2100
Dave	2200
Tina	2300
Ringo	1900

- The problems with our prototype software are from keeping two lists to track employees' salaries.

The Solution: Classes and Objects

- But we know how to combine them using classes and objects.
- We can think of a class as a blueprint to make an object.

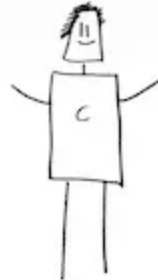


Creating the Employee Class

- We have a real-world object: an employee.
- We can make a class that abstracts an employee.
 - We need to abstract from the attributes of the object: name and salary.



Vera
\$2000



Chuck
\$1800



Dave
\$2200

```
class Employee:
    def __init__(self, name, salary):
        self.name = name
        self.salary = salary
```

Java vs Python: Class Definition

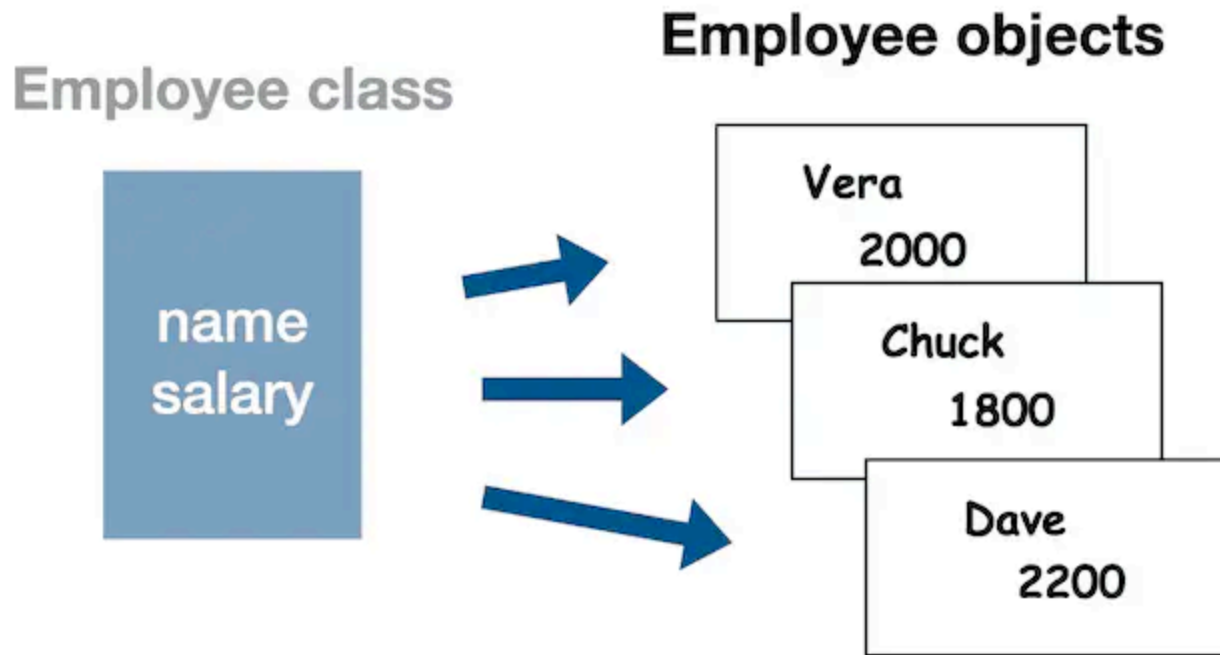
- We can use any OOP language for abstraction.
- For example, we can use Java to express the same class.
- Java is a typed language, so we need to add data types.

```
public class Employee { // Java
    String name;
    int salary;
    public Employee(String name, int salary) {
        this.name = name;
        this.salary = salary;
    }
}
```

```
class Employee: # Python
    def __init__(self, name, salary):
        self.name = name
        self.salary = salary
```

| Objects

- To use the class, we need to instantiate the class to create an object.
- The instantiated class is called an object.
- We store each object in a list in this example.



```
employees = [  
    Employee("Vera", 2000),  
    Employee("Chuck", 1800),  
    Employee("Dave", 2200),  
]
```

Java vs Python: Lists and Collections

- In this example, Python's list syntax is more concise than Java's typed collection declaration.

```
List<Employee> employees = new ArrayList<Employee>();  
employees.add(new Employee("Vera", 2000));  
employees.add(new Employee("Chuck", 1800));  
employees.add(new Employee("Dave", 2200));
```

Dynamic vs Static Languages

- **Dynamic Languages (e.g., Python):**
 - Rapid prototyping, concise syntax, fast iterative development.
 - Types are bound dynamically; runtime tests and type hints help catch errors.

Static Languages (e.g., Java, C++):

- Compiler catches type mismatches before execution.
- Static typing enables certain compiler optimizations, which is one reason (not the only one) many performance-critical systems use static languages.

Abstraction of Employee

```
1  employees = [  
2      Employee("Vera", 2000),  
3      Employee("Chuck", 1800),  
4      Employee("Samantha", 1800),  
5      Employee("Roberto", 2100),  
6      Employee("Dave", 2200),  
7      Employee("Tina", 2300),  
8      Employee("Ringo", 1900)  
9  ]  
10  for e in employees:  
11      print(f"{e.name}, ${e.salary}")
```



Vera, \$2000
Chuck, \$1800
Samantha, \$1800
Roberto, \$2100
Dave, \$2200
Tina, \$2300
Ringo, \$1900

Change, Again

- Mr. Star likes the program, and he now wants to add the feature of printing job titles.

NAME	SALARY	JOB TITLE
Vera	2000	Manager
Chuck	1800	Attendant
Samantha	1800	Attendant
Roberto	2100	Cook
Dave	2200	Car Repair
Tina	2300	Car Repair
Ringo	1900	Car Repair



```
Vera, $2000, Manager
Chuck, $1800, Attendant
Samantha, $1800, Attendant
Roberto, $2100, Cook
Dave, $2200, Car repair
Tina, $2300, Car repair
Ringo, $1900, Car repair
```

Requirements Version 3

- After some discussion, we (software engineers) and Mr. Star (a client) agreed upon new requirements.
- We have a high-level requirement (an **epic**, a large user story) with a smaller **sub-requirement** (a more specific user story) underneath it.

Epic:

As a "manager,"

I want to "see a list of the employees with their salaries,"
so that "I can manage my employees."

Sub-requirement 1:

As a "manager,"

I want to "have the list including name, salary, and job title,"
so that "I can track my employees effectively."

Quick Fix (Bad Solution)

```
1 for e in employees:
2     if e.name == "Vera":
3         print(f"{e.name}, ${e.salary}, Manager")
4     if e.name == "Chuck" or e.name == "Samantha":
5         print(f"{e.name}, ${e.salary}, Attendant")
6     if e.name == "Roberto":
7         print(f"{e.name}, ${e.salary}, Cook")
8     if e.name == "Dave" or e.name == "Tina" or e.name == "Ringo":
9         print(f"{e.name}, ${e.salary}, Car repair")
```

```
Vera, $2000, Manager
Chuck, $1800, Attendant
Samantha, $1800, Attendant
Roberto, $2100, Cook
Dave, $2200, Car repair
Tina, $2300, Car repair
Ringo, $1900, Car repair
```

Having multiple if statements means introducing complexity unnecessarily.

Code Smell: Job Title Determined by Employee Name

- The job title is guessed from the employee's name, instead of being part of the employee's own data.
- This is a quick and dirty approach that adds unnecessary complexity, not design update to solve problems.

- Every time an employee changes, we must update both the employee list and this if/else logic.

Problem indicators

Duplicate code

Coupling

No Single Responsibility

~~if/else~~

| The Two-Place Problem

- The employees have to be managed in **two places**.
- When there is a change, we should update the if/else statement and the list.

- We already made this mistake in the prototype development.

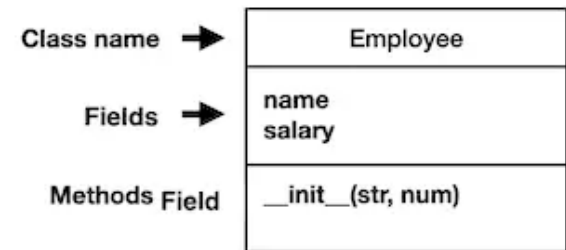
```
employees = [  
    Employee("Vera", 2000),  
    Employee("Chuck", 1800),  
    Employee("Samantha", 1800),  
    Employee("Roberto", 2100),  
    Employee("Dave", 2200),  
    Employee("Tina", 2300),  
    Employee("Ringo", 1900),  
]  
  
for e in employees:  
    if e.name == "Vera":  
        print(f"{e.name}, ${e.salary}, Manager")  
    if e.name == "Chuck" or e.name == "Samantha":  
        print(f"{e.name}, ${e.salary}, Attendant")  
    if e.name == "Roberto":  
        print(f"{e.name}, ${e.salary}, Cook")  
    if e.name == "Dave" or e.name == "Tina" or e.name == "Ringo":  
        print(f"{e.name}, ${e.salary}, Car repair")
```

**employees
have to be
managed in
two places**

Abstraction as a Solution

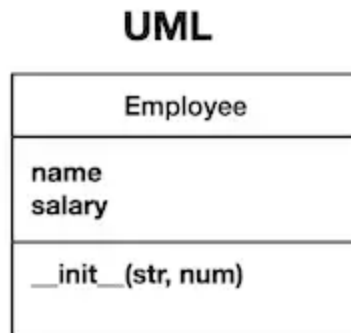
- We use the OOP abstraction technique (classes and objects) and can solve this issue.
- The following UML class diagram shows the abstraction of the Employee.

- The UML has three compartments: name, attributes (fields), and operations (methods).



UML To Python Code

- We can translate the UML class diagram into any OOP code, including Python.



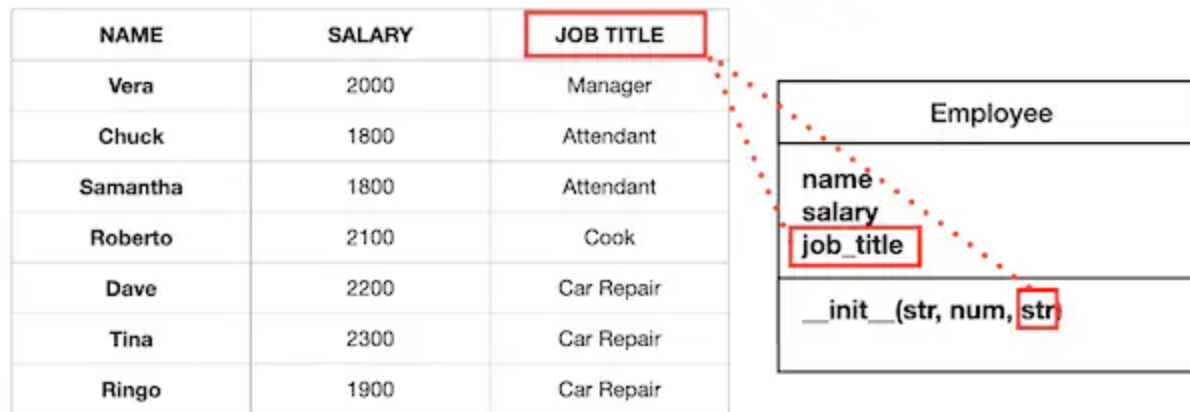
Python

```
class Employee:
    def __init__(self, name, salary):
        self.name = name
        self.salary = salary
```

Refactor design first

- We can manage the complexity with abstractions.
- Instead of updating code directly to meet requirements, we refactor our design (in this case, the UML class diagram).

- By analyzing a new requirement and our current design, we can identify that we need to add a new field to our class design.



Then, refactor the Code

```
class Employee:
    def __init__(self, name, salary, job_title):
        self.name = name
        self.salary = salary
        self.job_title = job_title

employees = [
    Employee("Vera", 2000, "Manager"),
    Employee("Chuck", 1800, "Attendant"),
    Employee("Samantha", 1800, "Attendant"),
    Employee("Roberto", 2100, "Cook"),
    Employee("Dave", 2200, "Car Repair"),
    Employee("Tina", 2300, "Car Repair"),
    Employee("Ringo", 1900, "Car Repair")
]
```

Clean Implementation

- With a slight modification, we can show the same results.

```
for e in employees:  
    print(f"{e.name}, ${e.salary}, {e.job_title}")
```

```
Vera, $2000, Manager  
Chuck, $1800, Attendant  
Samantha, $1800, Attendant  
Roberto, $2100, Cook  
Dave, $2200, Car repair  
Tina, $2300, Car repair  
Ringo, $1900, Car repair
```

Code Smell - Duplicate Code

- However, we can sense another code smell.
- We find that job titles are duplicated in many places.

- This means we should modify multiple places with a single change.
- How can we solve this problem in an OOP way?

```
class Employee:
    def __init__(self, name, salary, job_title):
        self.name = name
        self.salary = salary
        self.job_title = job_title

employees = [
    Employee("Vera", 2000, "Manager"),
    Employee("Chuck", 1800, "Station Attendant"),
    Employee("Samantha", 1800, "Station Attendant"),
    Employee("Roberto", 2100, "Cook"),
    Employee("Dave", 2200, "Mechanic"),
    Employee("Tina", 2300, "Mechanic"),
    Employee("Ringo", 1900, "Mechanic"),
]

for e in employees:
    print(f"{e.name}, ${e.salary}, {e.job_title}")
```

Problem indicators

~~Duplicate code~~

Coupling

No Single Responsibility

~~if/else~~

Checkpoint: Lessons on Abstraction

- OOP's abstraction allows us to manage complexity from changes.
- Whenever we need to change our code, we do not change it in a hurry.

IMPORTANT

- Instead, we pause, improve the class design, and then update the code.
- **Key Process:** Update the design and implementation together before changing behavior.

Duplication Calls for Refactoring.
Is Inheritance a Good Fit?

DRY Rule

- We have a software design rule, **DRY (Don't Repeat Yourself)**.
- Duplication means multiple impacts from a change.

- If Mr. Star changes the job title, it will impact multiple places in the code.

NAME	SALARY	JOB TITLE
Vera	2000	Manager
Chuck	1800	Station Attendant
Samantha	1800	Station Attendant
Roberto	2100	Cook
Dave	2200	Mechanic
Tina	2300	Mechanic
Ringo	1900	Mechanic



| Identifying Duplication

- The first step to solving this issue is identifying the duplication.
- In this example, `job_title` is duplicated.

- We can solve this duplication issue by creating new job classes.

Employee
name salary job_title
__init__(str, num, str)

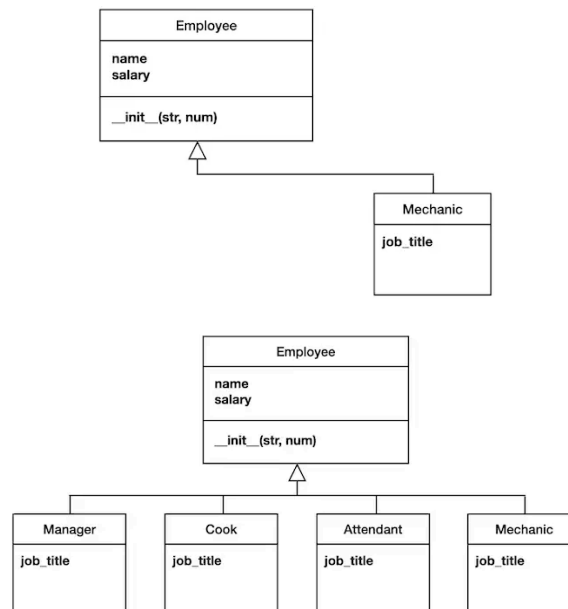
Mechanic
job_title

Choosing Inheritance

- Manager, Cook, Attendant, and Mechanic are all specific kinds of Employee — a valid **is-a** relationship.

- Note: In this redesign, we also normalized informal titles (e.g., "Car Repair" becomes "Mechanic", and "Attendant" is displayed as "Station Attendant").
- Each subclass reuses Employee's `name` and `salary`, inheriting shared attributes while defining its specific role.

- We define subclasses of **Employee** for specific job roles, such as Manager, Cook, Attendant, or Mechanic.

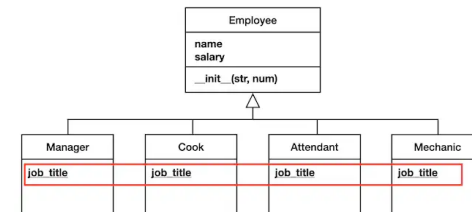


| Static Fields (Class Attributes)

- The job title of the class is the same for all the objects.
- For example, the job title will always be "Mechanic" for the Mechanic class.
- In Python, we call this a **class attribute**. In UML, the equivalent concept is a **static field**.

Implementation

```
1 class Employee:
2     def __init__(self, name, salary):
3         self.name = name
4         self.salary = salary
5
6 class Mechanic(Employee):
7     job_title = "Mechanic"
8 class Attendant(Employee):
9     job_title = "Station Attendant"
10 class Cook(Employee):
11     job_title = "Cook"
12 class Manager(Employee):
13     job_title = "Manager"
```



| The Rule of Three: When to Refactor

- If the same duplication appears three times, it is a refactoring signal.
- The Rule of Three tells us *when* to refactor, not *which* design to use.
- Duplication is a clue to rethink the design, not proof that inheritance is always the answer.

IMPORTANT

Choosing Inheritance: Is-A vs. Alternatives

- We choose **inheritance** because Manager, Cook, Attendant, and Mechanic are real Employee types.
- Inheritance is a good fit when there is a true **is-a** relationship.
- If we only had repeated labels, constants, an **Enum**, or a lookup table would be enough.

Benefits of using inheritance

- With inheritance, we can reuse existing code.
- We only extend (add) or override (revise) the missing features.

Tool box

✓ Objects & Classes

✓ Inheritance

Encapsulation

Polymorphism

Composition

Implementation

```
employees = [  
    Manager("Vera", 2000),  
    Attendant("Chuck", 1800),  
    Attendant("Samantha", 1800),  
    Cook("Roberto", 2100),  
    Mechanic("Dave", 2200),  
    Mechanic("Tina", 2300),  
    Mechanic("Ringo", 1900),  
]  
  
for e in employees:  
    print(f"{e.name}, ${e.salary}, {e.job_title}")
```

Problem solved

- **Success:** We can change **only one place** to change the job title!

```
Vera, $2000, Manager  
Chuck, $1800, Station Attendant  
Samantha, $1800, Station Attendant  
Roberto, $2100, Cook  
Dave, $2200, Mechanic  
Tina, $2300, Mechanic  
Ringo, $1900, Mechanic
```

Separation of Concerns

- In the code, we can separate our concerns into three sections: classes, creating data (population), and reporting.

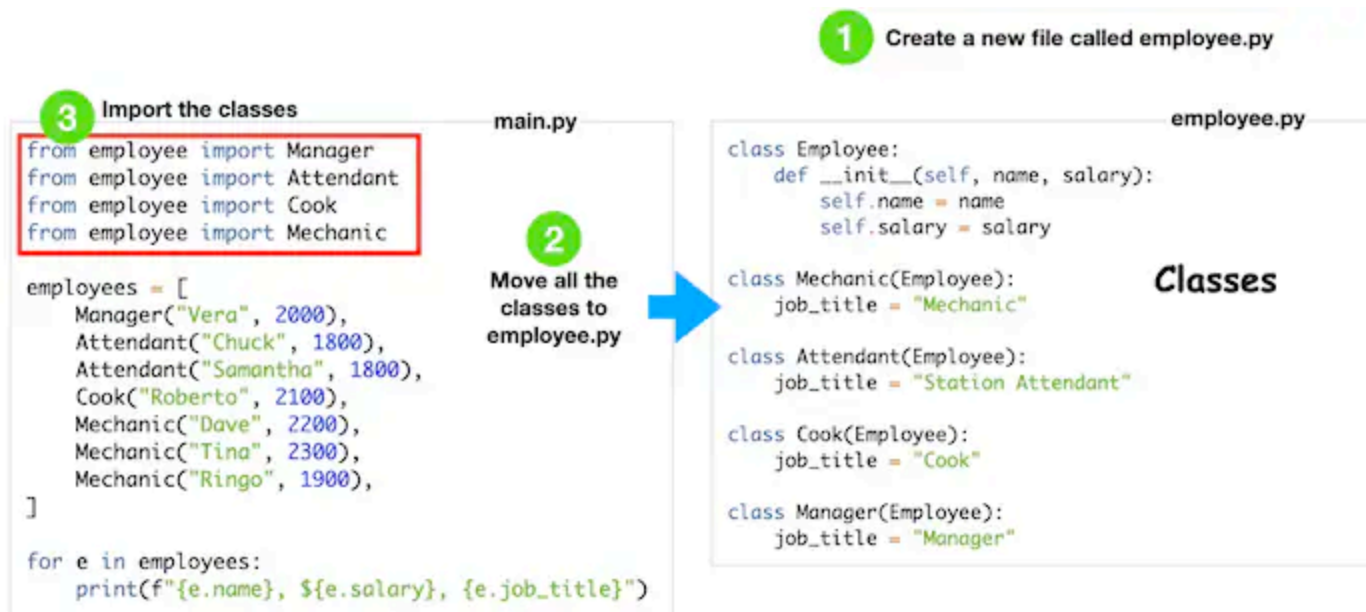
- Then, we can separate into different modules with clear boundaries to manage complexity from changes.

<pre>class Employee: def __init__(self, name, salary): self.name = name self.salary = salary class Mechanic(Employee): job_title = "Mechanic" class Attendant(Employee): job_title = "Station Attendant" class Cook(Employee): job_title = "Cook" class Manager(Employee): job_title = "Manager"</pre>	Classes
<pre>employees = [Manager("Vera", 2000), Attendant("Chuck", 1800), Attendant("Samantha", 1800), Cook("Roberto", 2100), Mechanic("Dave", 2200), Mechanic("Tina", 2300), Mechanic("Ringo", 1900)]</pre>	Create data
<pre>for e in employees: print(f"{e.name}, \${e.salary}, {e.job_title}")</pre>	Report

Module Organization

- We create `employee.py` for the 'Classes' section.
- We create `main.py` for the 'Create data' and 'Report' sections.

- The main.py uses the `from employee import` `...` to import classes in the employee.py module.



Module Relationship

- The `main.py` module **uses** the `employee.py` module, and an arrow is used to describe this relationship in UML.



Lessons Learned

- We should isolate the duplicated fields or methods when we see duplications.
- We can isolate them using **inheritance** when a genuine **is-a** relationship exists.

- As we refactor, we should organize code to separate concerns so that we can make well-defined modules.
- **Key Principle:** Inheritance can reuse genuinely shared state and behavior when a valid is-a relationship exists — it is one tool for removing duplication, not the only one.